FinAgent-Evo：面向金融投研的自主进化动态技能编排框架¶

研究提案¶

摘要（Executive Summary）¶

FinAgent-Evo 是一个创新的自主进化动态技能编排框架，专为金融投资研究场景设计。该框架通过融合大型语言模型（LLM）的推理能力、进化算法的优化能力以及多智能体协作机制，实现了金融AI Agent技能的自主发现、动态组合与持续进化。

核心创新点：

1. 技能进化引擎：基于遗传算法的技能拓扑优化，支持技能组合的自动发现与迭代改进
2. 动态编排机制：根据市场环境与任务需求实时调整技能执行路径
3. 多层级记忆系统：融合 episodic memory 与 procedural memory 支持经验复用
4. 自我改进循环：通过执行反馈实现技能库的持续扩充与优化

预期贡献：为金融AI Agent设计提供新的方法论范式，突破静态技能配置的局限性，实现真正自主进化的智能投研系统。

1. 问题定义与研究背景（Problem Definition）¶

1.1 金融投研AI Agent的现状与挑战¶

近年来，基于大型语言模型的金融交易Agent取得了显著进展。Li et al. (2024) 提出的 **TradingAgents** 框架模拟真实交易公司的协作动态，通过专业分析师角色（基本面、情绪、技术面）的协作实现投资决策[^1]。Yu et al. (2024) 的 **FinCon** 系统引入概念性语言强化机制增强金融决策能力[^2]。

然而，现有研究存在以下关键局限：

局限维度	具体问题	影响
静态技能配置	技能组合预设固定，无法适应市场变化	在非稳态市场环境中性能显著下降
单一任务导向	多针对特定交易任务优化，缺乏通用性	难以处理复杂的多步骤投研工作流
缺乏自主进化	依赖人工设计技能，无法自我改进	维护成本高，扩展性差
编排效率低下	简单串行或并行执行，未考虑技能间依赖关系	执行路径冗余，响应延迟高

[^1]: TradingAgents: Multi-Agents LLM Financial Trading Framework, arXiv:2412.20138, 2024. https://arxiv.org/abs/2412.20138
[^2]: FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement, NeurIPS 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/f7ae4fe91d96f50abc2211f09b6a7e49-Abstract-Conference.html

1.2 研究空白（Research Gap）¶

现有文献在以下方面存在明显空白：

1. 技能动态编排：虽然 Liu et al. (2026) 的 **SkillOrchestra** 提出了技能感知编排框架[^3]，但主要针对通用Agent路由，未考虑金融领域的特殊需求（如时序数据敏感性、风险约束等）。
2. 自主技能进化：Zheng et al. (2025) 的 **SkillWeaver** 实现了Web Agent的自主技能发现[^4]，但缺乏对金融技能（如技术分析、风险评估）的专门优化。
3. 进化算法与**Agent**架构搜索：Winter & Teahan (2025) 的 **ENAS** 和 Yuan (2025) 的研究[^5]展示了进化算法在神经架构搜索中的潜力，但尚未应用于Agent技能拓扑优化。

[^3]: SkillOrchestra: Learning to Route Agents via Skill Transfer, arXiv:2602.19672, 2026. https://arxiv.org/abs/2602.19672
[^4]: SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills, arXiv:2504.07079, 2025. https://arxiv.org/abs/2504.07079
[^5]: Ecological Neural Architecture Search, arXiv:2503.10908, 2025. https://arxiv.org/abs/2503.10908

1.3 研究问题¶

基于上述分析，本研究聚焦以下核心问题：

> **RQ1**：如何设计一种技能表示方法，既能捕捉金融投研任务的专业特性，又支持进化操作（变异、交叉、选择）？
>
> **RQ2**：如何构建动态编排机制，根据市场环境与任务特征实时优化技能执行路径？
>
> **RQ3**：如何建立自我改进循环，使Agent能够从执行历史中自主发现新技能并优化现有技能？
>
> **RQ4**：如何验证自主进化技能编排相较于静态配置在金融风险调整收益上的优势？

2. 相关工作（Related Work）¶

2.1 金融LLM Agent¶

多**Agent**协作框架：近期研究普遍采用多Agent架构模拟真实交易团队。Liu et al. (2026) 提出细粒度任务分解的多Agent系统，证明细粒度设计显著提升风险调整收益[^6]。Wang et al. (2024) 的 **StockAgent** 通过模拟投资者行为分析外部因素对交易的影响[^7]。

记忆与推理机制：Xie et al. (2023) 的 **FinMem** 引入分层记忆模块，与人类交易者的认知结构对齐[^8]。Li et al. (2024) 提出事实-主观感知推理框架，分离事实分析与主观判断[^9]。

[^6]: Toward Expert Investment Teams: A Multi-Agent LLM System with Fine-Grained Trading Tasks, arXiv:2602.23330, 2026. https://arxiv.org/abs/2602.23330
[^7]: StockAgent: LLM-based Stock Trading in Simulated Real-world Environments, arXiv: 2407.18957, 2024. https://arxiv.org/abs/2407.18957
[^8]: FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory, arXiv: 2311.13743, 2023. https://arxiv.org/abs/2311.13743
[^9]: Enhancing LLM Trading Performance with Fact-Subjectivity Aware Reasoning, arXiv: 2410.12464, 2024. https://arxiv.org/abs/2410.12464

2.2 技能编排与工具学习¶

技能编排框架：Liu et al. (2026) 的 **AgentSkillOS** 提出基于DAG的技能编排，支持大规模技能生态管理[^10]。该框架通过能力树组织技能，并实现树形检索近似Oracle选择效果。

技能发现与学习：Liu et al. (2025) 的 **PSEC** 框架[^11] 提出参数空间中的技能扩展与组合，使用LoRA模块实现高效技能迁移。Yang et al. (2024) 的 **Choreographer**[^12] 在想象环境中学习与适应技能。

[^10]: Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale, arXiv:2603.02176, 2026. https://arxiv.org/abs/2603.02176
[^11]: Skill Expansion and Composition in Parameter Space, arXiv:2502.05932, 2025. https://arxiv.org/abs/2502.05932
[^12]: Choreographer: Learning and Adapting Skills in Imagination, arXiv:2211.13350, 2024. https://arxiv.org/abs/2211.13350

2.3 进化算法与架构搜索¶

神经架构搜索：Y. Liu et al. (2021) 的综述[^13] 系统总结了进化神经架构搜索（ENAS）方法。Lu et al. (2019) 的 **NSGA-Net**[^14] 引入多目标遗传算法平衡准确率与复杂度。

多**Agent**架构优化：Shi et al. (2023) 的 **MANAS**[^15] 将NAS形式化为多Agent问题，实现O(√T)的遗憾界。Garcia 提出的 **Evo-NAS**[^16] 结合进化与神经Agent优势，在1/3搜索成本下达到更高准确率。

[^13]: A Survey on Evolutionary Neural Architecture Search, IEEE TNNLS, 2021. https://arxiv.org/abs/2008.10937
[^14]: NSGA-Net: Neural Architecture Search using Multi-Objective Genetic Algorithm, GECCO 2019. https://openreview.net/forum?id=B1gIf305Ym
[^15]: MANAS: Multi-Agent Neural Architecture Search, arXiv:1909.01051, 2023. https://arxiv.org/abs/1909.01051
[^16]: Evolutionary-Neural Hybrid Agents for Architecture Search, arXiv:1811.09828, 2018. https://arxiv.org/abs/1811.09828

2.4 多**Agent**强化学习在金融中的应用¶

投资组合管理：Zhang et al. (2020) 的 **MAPS**[^17] 提出基于多Agent强化学习的投资组合管理系统，通过多样化策略降低风险。

做市策略：Vicente (2025) 的博士论文[^18] 系统研究了RL在高频做市中的应用，提出POW-dTS算法实现策略动态选择。

市场模拟：Lussange et al. (2021-2024)[^19][^20] 系列工作使用多Agent RL建模股票市场，验证Agent学习行为对市场动态的影响。

[^17]: MAPS: Multi-agent Reinforcement Learning-based Portfolio Management System, arXiv:2007.05402, 2020. https://arxiv.org/abs/2007.05402
[^18]: Market Making Strategies with Reinforcement Learning, arXiv:2507.18680, 2025. https://arxiv.org/abs/2507.18680
[^19]: Modelling Stock Markets by Multi-Agent Reinforcement Learning, Computational Economics, 2021. https://hal.science/hal-03055070
[^20]: Mesoscale Effects of Trader Learning Behaviors in Financial Markets, PLOS ONE, 2024. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0301141

3. 提出的方法（Proposed Method）¶

3.1 框架概述¶

FinAgent-Evo 采用分层架构设计，包含四个核心模块：

```
┌──────────────
```

3.2 技能表示与编码（Skill Representation）¶

3.2.1 技能基因型编码¶

每个技能表示为基因型-表现型映射结构：

```
SkillGenotype = {
    "skill_id": str,           # 唯一标识
    "category": Enum,          # 技能类别：DATA|ANALYSIS|DECISION|EXECUTION
    "llm_config": {            # LLM配置基因
        "model_tier": Enum,    # LIGHT|STANDARD|HEAVY
        "temperature": float,  # 采样温度
        "max_tokens": int,     # 最大生成长度
    },
    "prompt_chromosome": str,  # 提示模板（可进化）
    "tool_deps": [str],        # 依赖工具列表
    "input_schema": Dict,      # 输入数据模式
    "output_schema": Dict,     # 输出数据模式
    "fitness_score": float,    # 适应度评分
    "execution_history": List  # 执行历史记录
}
```

3.2.2 技能拓扑图（Skill Topology Graph）¶

技能组合表示为有向无环图（DAG）：

$$G = (V, E, W)$$

其中：
- V：技能节点集合
- E：执行依赖边
- W：边权重（数据流特征）

3.3 技能进化引擎（Skill Evolution Engine）¶

3.3.1 进化操作算子¶

1. 变异算子（Mutation）

```
def mutate(skill: SkillGenotype) -> SkillGenotype:
    """
    支持以下变异类型：
     - prompt_mutation: 提示模板词语替换/增删
     - param_mutation: LLM参数微调
     - structure_mutation: 输入/输出模式变更
    """
     mutation_type = sample(['prompt', 'param', 'structure'],
                            p=[0.6, 0.3, 0.1])
     if mutation_type == 'prompt':
         return prompt_mutation(skill)
     elif mutation_type == 'param':
         return param_mutation(skill)
     else:
         return structure_mutation(skill)
```

2. 交叉算子（Crossover）

采用基于序列对齐的交叉策略，借鉴 Smith-Waterman 算法[^21]：

```
def crossover(parent1: SkillGenotype,
              parent2: SkillGenotype) -> Tuple[SkillGenotype, SkillGenotype]:
    """
    基于提示模板序列相似性的交叉操作
    """
     alignment = smith_waterman_align(
         parent1.prompt_chromosome,
         parent2.prompt_chromosome
    )
     child1, child2 = template_recombine(parent1, parent2, alignment)
     return child1, child2
```

[^21]: Evolutionary Architecture Search through Grammar-Based Sequence Alignment, arXiv: 2512.04992, 2025. https://arxiv.org/abs/2512.04992

3. 选择策略（Selection）

采用锦标赛选择结合精英保留：

$$P_{select}(s_i) = \frac{f(s_i)^\tau}{\sum_j f(s_j)^\tau}$$

其中 τ 为选择压力参数，$f(s_i)$ 为技能 i 的适应度。

3.3.2 适应度评估¶

技能适应度采用多目标评估：

$$F(s) = w_1 \cdot R_{sharpe} + w_2 \cdot (-MDD) + w_3 \cdot E_{efficiency} - w_4 \cdot C_{cost}$$

其中：
- R_{sharpe}：策略夏普比率
- MDD：最大回撤
- $E_{efficiency}$：执行效率（任务完成率/延迟）
- C_{cost}：API调用成本
- w_i：权重系数

3.4 动态编排机制（Dynamic Orchestration）¶

3.4.1 上下文感知路由¶

编排器基于当前市场状态 M_t 和任务目标 T 选择最优技能路径：

$$\pi^* = \arg\max_{\pi \in \Pi} \mathbb{E}[R(\pi | M_t, T)]$$

其中 Π 为所有可行路径集合。

3.4.2 基于注意力的技能选择¶

引入多头注意力机制计算技能相关性：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Query Q：当前任务嵌入
- Key K：技能描述嵌入
- Value V：技能执行价值估计

3.4.3 自适应重规划¶

当执行偏差超过阈值时触发重规划：

```
def adaptive_replan(current_state, original_plan):
    deviation = compute_deviation(current_state, original_plan)
    if deviation > threshold:
       # 触发局部重规划
        return local_replan(current_state, original_plan)
    return original_plan
```

3.5 分层记忆系统（Hierarchical Memory）¶

借鉴 FinMem[^8] 的分层记忆设计，构建三级记忆结构：

记忆层级	时间跨度	内容	检索方式
工作记忆	当前会话	临时上下文、中间结果	全量保留
情景记忆	近期历史	具体交易案例、市场事件	相似度检索
程序记忆	长期积累	抽象策略、经验规则	模式匹配

记忆写入与检索算法：

```
class HierarchicalMemory:
    def write(self, experience: Experience):
      # 写入工作记忆
        self.working_memory.append(experience)

      # 重要经验升级至情景记忆
        if experience.importance > θ_episodic:
            self.episodic_memory.store(experience)

      # 模式抽象至程序记忆
        patterns = self.abstract_patterns(self.episodic_memory)
        self.procedural_memory.update(patterns)

    def retrieve(self, query: Query, k: int = 5) -> List[Experience]:
      # 多级检索
        working = self.working_memory.get_recent()
        episodic = self.episodic_memory.similarity_search(query, k)
        procedural = self.procedural_memory.pattern_match(query)
        return merge_and_rank(working, episodic, procedural)
```

3.6 自我改进循环（Self-Improvement Loop）¶

FinAgent-Evo 的核心创新在于建立完整的自我改进闭环：

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                 自我改进循环                                                                 │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                                                              │
│                                                                                                                              │
│    ┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐                                                  │
│    │  任务执行  │──────→│  效果评估  │──────→│  经验提取  │                                                  │
│    │ Execute  │     │ Evaluate │     │ Extract  │                                                  │
│    └──────────────────┘     └──────────────────┘     └──────────────────┘                                                  │
│                                  │                    │                    │                                                  │
│                                  ▼                    ▼                    │                                                  │
│                              ┌──────────────────┐     ┌──────────────────┐                                                  │
│                              │ 适应度更新 │     │ 新模式发现 │                                                  │
│                              │ Update   │     │ Discover │                                                  │
│                              │ Fitness  │     │ Pattern  │                                                  │
│                              └──────────────────┘     └──────────────────┘                                                  │
│                                  │                    │                    │                                                  │
│                                  └──────────────────────────────────────┘                                                  │
│                                                  ▼                                                                           │
│                                              ┌──────────────────┐                                                            │
│                                              │ 进化迭代 │                                                            │
│                                              │ Evolve   │◀──────────────────────┐                                            │
│                                              │ Skills   │   触发条件满足   │                                            │
│                                              └──────────────────┘                                                            │
│                                                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

4. 实验计划（Experiment Plan）¶

4.1 数据集与基准¶

数据集	描述	用途
InvestorBench[^22]	首个LLM金融决策基准	综合性能评估
NASDAQ-100	纳斯达克100成分股	股票交易实验
Crypto-Binance	加密货币交易数据	高频交易测试
Financial News	财经新闻与公告	情绪分析技能评估

[^22]: INVESTORBENCH: A Benchmark for Financial Decision-Making Tasks, arXiv:2412.18174, 2024. https://arxiv.org/abs/2412.18174

4.2 评估指标¶

性能指标：

- 累积收益率（**Cumulative Return**）: $R = \prod_{t=1}^{T}(1 + r_t) - 1$
- 夏普比率（**Sharpe Ratio**）: $SR = \frac{\bar{r} - r_f}{\sigma_r}$
- 最大回撤（**Max Drawdown**）: $MDD = \max_{\tau} \left[\max_{s \leq \tau} \left(\frac{X_s - X_\tau}{X_s} \right) \right]$
- **Calmar**比率: $CR = \frac{\bar{r}}{MDD}$

效率指标：

- 平均任务完成延迟
- API调用次数/成本
- 技能编排路径长度

进化指标：

- 技能库规模增长率
- 技能适应度提升曲线
- 新技能发现频率

4.3 基线方法¶

方法	类型	来源
Buy & Hold	传统策略	市场基准
FinMem	单Agent	Xie et al., 2023[^8]
TradingAgents	多Agent	Li et al., 2024[^1]
SkillOrchestra	技能编排	Liu et al., 2026[^3]
FinCon	多Agent强化	Yu et al., 2024[^2]

4.4 主实验（**Main Experiments**）¶

实验1：端到端交易性能对比¶

目标：验证FinAgent-Evo在真实市场数据上的投资表现

设置：

- 回测期间：2020-01-01 至 2024-12-31

- 资产池：NASDAQ-100成分股
- 初始资金：$100,000
- 再平衡频率：日度/周度

预期结果：

- FinAgent-Evo的夏普比率较TradingAgents提升 ≥ 15%
- 最大回撤控制在 15% 以内
- 风险调整收益显著优于Buy & Hold

实验2：技能进化效果评估¶

目标：验证自主进化机制的有效性

设置：

- 对照组：固定技能配置（无进化）
- 实验组：启用技能进化引擎
- 进化代数：50代
- 种群规模：100

预期结果：

- 50代后平均技能适应度提升 ≥ 30%
- 新发现的有效技能 ≥ 10个
- 进化后策略对市场变化的适应性显著增强

实验3：动态编排效率分析¶

目标：评估动态编排相较静态编排的效率优势

设置：

- 任务类型：多步骤投研工作流（数据获取→分析→决策→执行）
- 对比：串行执行 vs 并行执行 vs 动态编排

预期结果：

- 任务完成时间减少 ≥ 25%
- API调用成本降低 ≥ 20%
- 任务成功率提升 ≥ 10%

4.5 消融实验（Ablation Studies）¶

实验	变体	目的
A1	无进化引擎	验证进化必要性
A2	无动态编排	验证动态编排必要性
A3	无分层记忆	验证记忆系统贡献
A4	简化适应度函数（单目标）	验证多目标优化必要性
A5	固定LLM配置	验证LLM配置进化价值

4.6 定性分析¶

案例研究：

1. 牛市场景：分析技能组合如何适应上涨趋势
2. 熊市场景：分析风险管控技能的自动激活
3. 黑天鹅事件：分析系统对突发事件的响应能力
4. 跨市场迁移：分析已进化技能在新市场的迁移能力

可解释性分析：

- 技能使用频率热力图
- 决策路径可视化
- 进化轨迹追踪

5. 风险分析与缓解策略（Risk Analysis）¶

5.1 技术风险¶

风险	可能性	影响	缓解策略
进化收敛到局部最优	中	高	引入多样性维持机制；多起点并行进化
技能组合爆炸	中	中	限制技能拓扑复杂度；引入剪枝机制
API成本过高	高	中	实现缓存机制；使用轻量模型进行初步筛选
过拟合历史数据	中	高	交叉验证；正则化适应度函数；引入市场模拟
延迟过高	中	高	预计算技能嵌入；异步编排优化

5.2 金融风险¶

风险	可能性	影响	缓解策略
市场非稳态导致技能失效	高	高	在线学习机制；定期重新进化
极端市场条件	低	极高	硬编码风险管控规则作为兜底
回测过拟合	高	高	前向验证；引入模拟交易期

5.3 实现风险¶

风险	可能性	影响	缓解策略
数据集获取困难	低	中	使用公开数据集；与数据提供商合作
计算资源不足	中	中	优化进化算法效率；使用云计算资源
LLM API限制	中	中	多提供商备用；本地模型降级方案

6. 时间规划与里程碑（Timeline）¶

阶段	时间	里程碑	交付物
Phase 1: 基础架构	1–2月	完成核心框架搭建	技能表示系统、基础编排器
Phase 2: 进化引擎	3–4月	实现完整进化机制	进化算子、适应度评估
Phase 3: 记忆与自改进	5–6月	集成自我改进循环	分层记忆、反馈机制
Phase 4: 实验验证	7–9月	完成全部实验	实验报告、对比分析
Phase 5: 优化完善	10–11月	系统优化与文档	最终系统、论文初稿
Phase 6: 论文发表	12月	投稿顶级会议/期刊	完整论文

---

7. 预期贡献（Expected Contributions）¶

7.1 理论贡献¶

1. 技能拓扑进化理论：提出面向LLM Agent的技能基因型–表现型映射模型
2. 动态编排优化框架：建立上下文感知的多技能组合优化理论
3. 金融**Agent**自进化理论：探索金融环境下自主Agent进化规律

7.2 技术贡献¶

1. 开源框架：发布FinAgent-Evo开源实现
2. 基准扩展：扩展InvestorBench基准，增加技能进化评估维度
3. 工具链：提供可视化工具集支持Agent行为分析

7.3 应用贡献¶

1. 金融**AI**方法论：为金融机构构建自适应AI系统提供方法论指导
2. 风险管控：提供具备自我改进能力的智能风控方案
3. 投研效率：提升投研自动化水平，降低人工成本

8. 参考文献（References）¶

1. Li, Y., et al. (2024). TradingAgents: Multi-Agents LLM Financial Trading Framework. *arXiv preprint* arXiv:2412.20138. https://arxiv.org/abs/2412.20138
2. Yu, Y., et al. (2024). FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making. *NeurIPS 2024*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/f7ae4fe91d96f50abc2211f09b6a7e49-Abstract-Conference.html
3. Liu, J., et al. (2026). SkillOrchestra: Learning to Route Agents via Skill Transfer. *arXiv preprint* arXiv:2602.19672. https://arxiv.org/abs/2602.19672
4. Zheng, B., et al. (2025). SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills. *arXiv preprint* arXiv:2504.07079. https://arxiv.org/abs/2504.07079
5. Winter, B.D., & Teahan, W.J. (2025). Ecological Neural Architecture Search. *arXiv preprint* arXiv:2503.10908. https://arxiv.org/abs/2503.10908
6. Liu, X., et al. (2026). Toward Expert Investment Teams: A Multi-Agent LLM System with Fine-Grained Trading Tasks. *arXiv preprint* arXiv:2602.23330. https://arxiv.org/abs/2602.23330
7. Wang, M., et al. (2024). StockAgent: LLM-based Stock Trading in Simulated Real-world Environments. *arXiv preprint* arXiv:2407.18957. https://arxiv.org/abs/2407.18957
8. Xie, Y., et al. (2023). FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design. *arXiv preprint* arXiv:2311.13743. https://arxiv.org/abs/2311.13743
9. Li, Y., et al. (2024). Enhancing LLM Trading Performance with Fact-Subjectivity Aware Reasoning. *arXiv preprint* arXiv:2410.12464. https://arxiv.org/abs/2410.12464
10. Liu, Y., et al. (2026). Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale. *arXiv preprint* arXiv:2603.02176. https://arxiv.org/abs/2603.02176
11. Liu, T., et al. (2025). Skill Expansion and Composition in Parameter Space. *arXiv preprint* arXiv:2502.05932. https://arxiv.org/abs/2502.05932
12. Yang, J., et al. (2024). Choreographer: Learning and Adapting Skills in Imagination. *arXiv preprint* arXiv:2211.13350. https://arxiv.org/abs/2211.13350

13. Liu, Y., et al. (2021). A Survey on Evolutionary Neural Architecture Search. *IEEE Transactions on Neural Networks and Learning Systems*. https://arxiv.org/abs/2008.10937

14. Lu, Z., et al. (2019). NSGA-Net: Neural Architecture Search using Multi-Objective Genetic Algorithm. *GECCO 2019*. https://openreview.net/forum?id=B1gIf305Ym

15. Shi, H., et al. (2023). MANAS: Multi-Agent Neural Architecture Search. *arXiv preprint* arXiv:1909.01051. https://arxiv.org/abs/1909.01051

16. Garcia, A. (2018). Evolutionary-Neural Hybrid Agents for Architecture Search. *arXiv preprint* arXiv:1811.09828. https://arxiv.org/abs/1811.09828

17. Zhang, Y., et al. (2020). MAPS: Multi-agent Reinforcement Learning-based Portfolio Management System. *arXiv preprint* arXiv:2007.05402. https://arxiv.org/abs/2007.05402

18. Vicente, O.F. (2025). Market Making Strategies with Reinforcement Learning. *arXiv preprint* arXiv:2507.18680. https://arxiv.org/abs/2507.18680

19. Lussange, J., et al. (2021). Modelling Stock Markets by Multi-Agent Reinforcement Learning. *Computational Economics*. https://hal.science/hal-03055070

20. Lussange, J., et al. (2024). Mesoscale Effects of Trader Learning Behaviors in Financial Markets. *PLOS ONE*. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0301141

21. Anonymous (2025). Evolutionary Architecture Search through Grammar-Based Sequence Alignment. *arXiv preprint* arXiv:2512.04992. https://arxiv.org/abs/2512.04992

22. Chen, Y., et al. (2024). INVESTORBENCH: A Benchmark for Financial Decision-Making Tasks with LLM-based Agent. *arXiv preprint* arXiv:2412.18174. https://arxiv.org/abs/2412.18174

提案版本：*v1.0*
最后更新：2026-03-26